

Case Study: AI-Powered Solar Energy Production Forecasting

Project Title:

Forecasting Solar Power Generation Using AI and Weather Data

Presented By :

Rohan Deshmukh

Sham Barsale

Aryan Kalsait

Ritesh Kakad

Objective:

To develop an AI-driven system that accurately predicts solar energy production using historical solar output and real-time weather conditions. This helps optimize energy grid management, storage, and consumption planning.

Problem Statement:

Solar energy production is **highly dependent on weather conditions**, which vary unpredictably. Energy providers struggle to integrate solar power effectively due to its **intermittent nature**. There is a strong need for a system that can **forecast solar power generation** with high accuracy.

Solution Overview:

An AI-powered forecasting system using machine learning, specifically **LSTM neural networks**, was implemented to predict hourly solar power generation. It leverages:

- Historical solar energy data
 - Weather parameters (temperature, humidity, irradiance, wind speed)
-

Dataset Used:

- **Source:** Kaggle (Solar Power Generation Data)
 - **Fields:** Timestamp, Solar Power Output (kWh), Temperature, Humidity, Wind Speed, Solar Irradiance
-

Technology Stack:

Component	Tool/Library
Programming Language	Python
ML Library	TensorFlow / Keras
Data Analysis	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Model Type	LSTM (Long Short-Term Memory)
Preprocessing	MinMaxScaler (Sklearn)

Process Workflow:

- Data Preprocessing**
 - Missing values handled using forward fill
 - Features normalized using MinMaxScaler
 - Time-series sequences created for LSTM input
 - Model Building**
 - An LSTM model with two layers was used
 - Trained to predict the next hour's solar power output
 - Evaluation Metrics**
 - Mean Absolute Error (MAE)
 - Root Mean Squared Error (RMSE)
 - Results Visualization**
 - Compared predicted vs actual values on test data
 - Plotted for visual assessment and reporting
-

Results:

- The model achieved an **RMSE of ~0.13** (on normalized values)
 - It was able to **predict solar output trends accurately**, with deviations mostly under extreme weather shifts
 - Prediction window:** 24-hour rolling forecast
 - The system can be updated with **real-time weather API** data for live forecasts
-

Key Learnings:

- LSTM is effective** for capturing temporal patterns in solar power data.
 - Weather features** like irradiance and humidity significantly affect model accuracy.
 - Data quality** and **feature scaling** play a major role in performance.
-

Impact:

- **Energy Providers:** Better grid load balancing
 - **Solar Farms:** Improved panel maintenance planning
 - **Smart Homes:** Efficient battery and device scheduling
-

Future Scope:

- Integrate real-time **weather APIs** (e.g., OpenWeatherMap)
- Build a **web dashboard** for interactive forecast visualizations
- Expand the system to **multi-site forecasting** for large-scale solar farms

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Load your data
data = pd.read_csv("solar_weather_data.csv", parse_dates=['timestamp'],
index_col='timestamp')
data = data[['solar_power_kwh', 'temperature', 'humidity', 'irradiance']] # adjust based on
your CSV

# Fill missing values
data.fillna(method='ffill', inplace=True)

# Normalize features
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)

# Create sequences for LSTM
def create_sequences(data, seq_length):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i+seq_length])
        y.append(data[i+seq_length][0]) # predicting solar_power_kwh
    return np.array(X), np.array(y)

seq_length = 24 # 24 hours
```

```

X, y = create_sequences(scaled_data, seq_length)

# Split into train/test
split = int(len(X) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Build LSTM Model
model = Sequential()
model.add(LSTM(64, return_sequences=True, input_shape=(X_train.shape[1],
X_train.shape[2])))
model.add(LSTM(32))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2)

# Predict and plot
predictions = model.predict(X_test)
predicted = scaler.inverse_transform(np.hstack((predictions, np.zeros((len(predictions),
scaled_data.shape[1] - 1)))))[:,0]
actual = scaler.inverse_transform(np.hstack((y_test.reshape(-1, 1), np.zeros((len(y_test),
scaled_data.shape[1] - 1)))))[:,0]

plt.plot(actual, label='Actual')
plt.plot(predicted, label='Predicted')
plt.legend()
plt.title("Solar Energy Forecasting")
plt.xlabel("Time Step")
plt.ylabel("Solar Power (kWh)")
plt.show()

```

Data Set Files in .csv :

[Dataset Link](#)

Use this link to download the dataset to use the model.

Github Link: <https://github.com/Rohan-2910/Final>